# Assignment and the Hungarian Method

Choose one worker for each job at minimum total cost.

> We will use the dual constraints to identify promising worker–job pairs, improve a partial assignment, and certify the final answer.

| Study sequence | What you should be able to do |
| --- | --- |
| **Model** | Write the assignment LP and its dual |
| **Hungarian method** | Find zero reduced-cost pairs, adjust the dual, and complete a matching |
| **Auction interpretation** | Explain how competing choices and changing job values resolve conflicts |

Worked practice answers and complete arguments follow their questions or appear in the linked appendix.

# Four technicians, four jobs

A company must complete four jobs. Each technician can do exactly one job, and each job needs exactly one technician.

The table gives the cost $c_{ij}$ of assigning worker $i$ to job $j$. All 16 assignments are allowed.

> Choose four entries: one in every row and one in every column. Minimize their sum.

## Original costs

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 10 | 19 | 8 | 15 |
| **W2** | 10 | 18 | 7 | 17 |
| **W3** | 13 | 16 | 9 | 14 |
| **W4** | 12 | 19 | 8 | 18 |

# From a whole assignment to an LP

Let $x_{ij}$ be the fraction of job $j$ assigned to worker $i$. For $n$ workers and $n$ jobs, a whole assignment uses 1 for a chosen pair and 0 otherwise.

$$\begin{aligned} \min \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij} \\ \text{s.t.} \quad & \sum_{j} x_{ij} = 1 \quad (\forall i), \\ & \sum_{i} x_{ij} = 1 \quad (\forall j), \\ & x_{ij} \geq 0. \end{aligned}$$

This continuous LP permits fractions, but an optimal whole assignment exists. The Hungarian method will construct one; the appendix explains why fractions cannot improve on it.

# Keep the assignment LP and its dual together

**Primal: assignment fractions**

$$\begin{aligned} \min \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij} \\ \text{s.t.} \quad & \sum_{j} x_{ij} = 1 \quad (\forall i), \\ & \sum_{i} x_{ij} = 1 \quad (\forall j), \\ & x_{ij} \geq 0. \end{aligned}$$

**Dual: one variable per equality**

$$\begin{aligned} \max \quad & \sum_{i} u_i + \sum_{j} v_j \\ \text{s.t.} \quad & u_i + v_j \leq c_{ij} \quad (\forall i, j), \\ & u_i,\ v_j \text{ unrestricted.} \end{aligned}$$

$u_i$ belongs to worker $i$'s row; $v_j$ belongs to job $j$'s column. The coefficient of $x_{ij}$ in the two equalities is 1, giving $u_i + v_j \leq c_{ij}$.

# Zero reduced cost identifies an active dual constraint

For any dual-feasible $u, v$, define $R_{ij} = c_{ij} - u_i - v_j \geq 0$. These are the **reduced costs**.

$$\sum_{i,j} c_{ij} x_{ij} = \sum_i u_i + \sum_j v_j + \sum_{i,j} R_{ij} x_{ij}.$$

The equality uses each worker-row sum and each job-column sum being 1.

> If a complete assignment uses only entries with reduced cost zero, its cost equals the dual value. Weak duality then proves both solutions optimal.

A zero entry means an active dual constraint. It does not mean that the original assignment is free.

# Start with a feasible dual: subtract row minima

Set each worker dual variable to the smallest cost in its original row. Initially every job dual variable is zero.

### Original costs

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 10 | 19 | 8 | 15 |
| **W2** | 10 | 18 | 7 | 17 |
| **W3** | 13 | 16 | 9 | 14 |
| **W4** | 12 | 19 | 8 | 18 |

### Subtract each row's minimum

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 2 | 11 | **0** | 7 |
| **W2** | 3 | 11 | **0** | 10 |
| **W3** | 4 | 7 | **0** | 5 |
| **W4** | 4 | 11 | **0** | 10 |

$$u = (8, 7, 9, 8), \qquad v = (0, 0, 0, 0).$$

For example, $R_{12} = 19 - 8 = 11$, while $R_{13} = 8 - 8 = 0$. Every reduced cost is nonnegative.

> Every complete assignment costs at least 8 + 7 + 9 + 8 = 32.

# Then subtract the remaining column minima

The column minima in the left table are (2, 7, 0, 5). Increase the job dual variables by these amounts.

**Before: reduced costs after row subtraction**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 2 | 11 | **0** | 7 |
| **W2** | 3 | 11 | **0** | 10 |
| **W3** | 4 | 7 | **0** | 5 |
| **W4** | 4 | 11 | **0** | 10 |

**After: also subtract column minima**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | **0** | 4 | **0** | 2 |
| **W2** | 1 | 4 | **0** | 5 |
| **W3** | 2 | **0** | **0** | **0** |
| **W4** | 2 | 4 | **0** | 5 |

$$u = (8, 7, 9, 8), \qquad v = (2, 7, 0, 5).$$

For example, $R_{14} = 15 - 8 - 5 = 2$. The dual lower bound increases from 32 to 46.

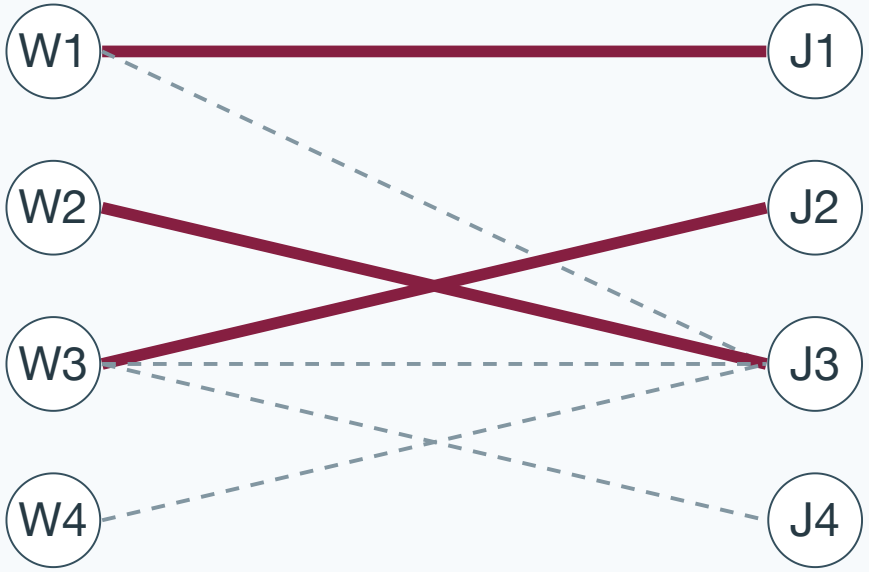
> The original assignment costs have not changed. Only the dual variables and reduced costs changed.

# A matching cannot reuse a worker or a job

A **matching** is a set of pairs with no worker or job repeated. It is **perfect** when all workers and all jobs are matched. A **maximum matching** has the largest possible number of pairs.

In the zero reduced-cost graph, draw a line only when the corresponding dual constraint is active.

> Several workers may want the same zero entry’s column. A zero in each row is therefore not enough: we must find distinct jobs.



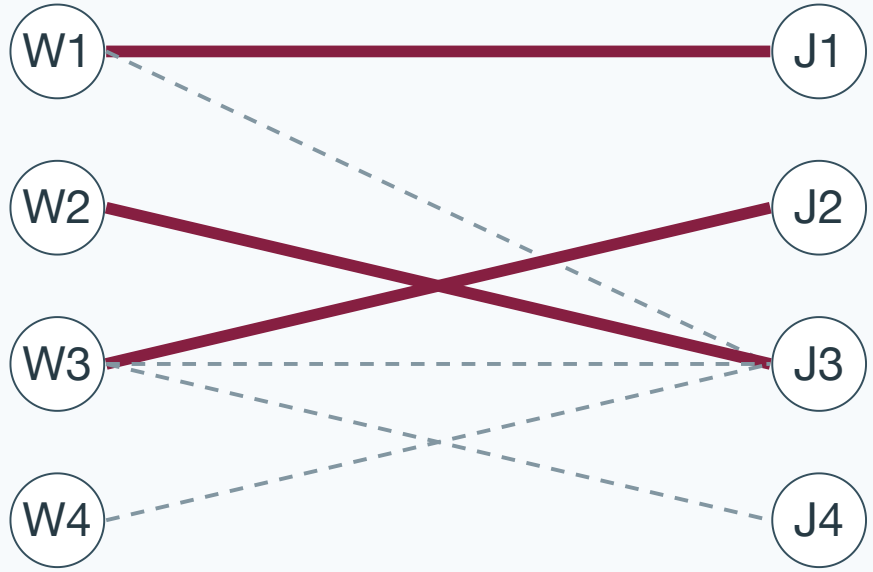

Every line has reduced cost zero. Thick maroon lines are chosen pairs; dashed lines are other available pairs.

# Three pairs work, but worker 4 is unmatched

**Reduced costs: original cost - worker dual variable - job dual variable**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | **0 ★** | 4 | **0** | 2 |
| **W2** | 1 | 4 | **0 ★** | 5 |
| **W3** | 2 | **0 ★** | **0** | **0** |
| **W4** | 2 | 4 | **0** | 5 |

★ marks W1–J1, W2–J3, W3–J2. These three pairs use different rows and columns.




Every line has reduced cost zero. Thick maroon lines are chosen pairs; dashed lines are other available pairs.

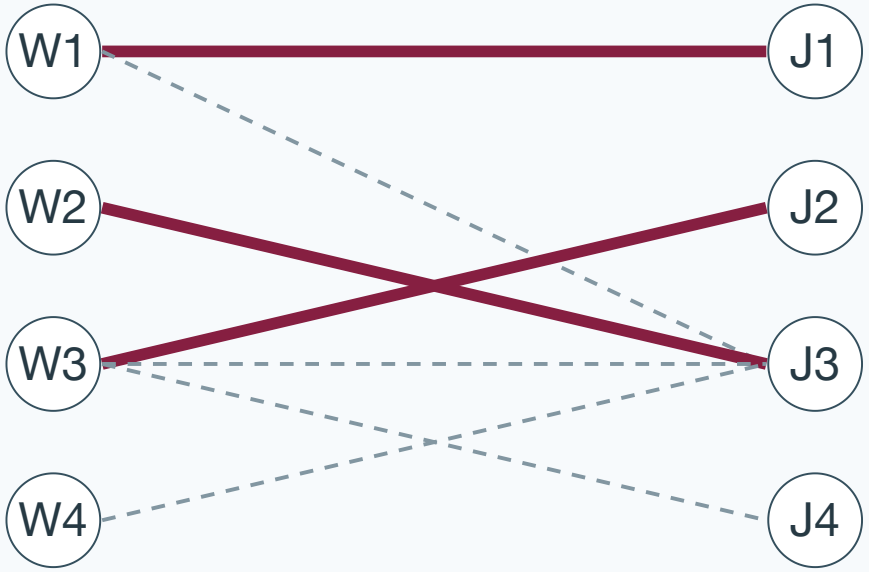
W2 and W4 both have only J3 available at reduced cost zero. They cannot both be matched. Thus no perfect zero matching exists yet.

# Search for a way to rearrange the chosen pairs

Start at an unmatched worker. Follow an unused zero edge to a job. If that job is already assigned, follow its chosen edge backward to its current worker.

> An **alternating path** switches between unused and chosen edges. If it reaches an unmatched job, exchange chosen and unused edges along the path. The matching grows by one.

Here: W4 → J3 → W2. W2 has no other zero edge, so the search cannot reach the unmatched job J4.




Every line has reduced cost zero. Thick maroon lines are chosen pairs; dashed lines are other available pairs.

# The failed search tells us which rows to adjust

Let S be reached workers and T be reached jobs. Here S = {W2, W4}, T = {J3}.

Every zero edge from a reached worker ends at a reached job; otherwise the search would continue.

> Cover the rows outside S (W1, W3) and the columns in T (J3). These three lines cover every zero.

The cover has three lines, equal to the matching size. This confirms that no larger zero matching exists.

**Shaded: covered rows W1, W3 and column J3**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | **0** | 4 | **0** | 2 |
| **W2** | 1 | 4 | **0** | 5 |
| **W3** | 2 | **0** | **0** | **0** |
| **W4** | 2 | 4 | **0** | 5 |

# Create a new zero without breaking dual feasibility

The uncovered entries lie in rows W2, W4 and columns J1, J2, J4. Choose θ as their smallest value, so subtracting θ cannot make an entry negative.

| Uncovered row | J1 | J2 | J4 |
|---|---|---|---|
| W2 | 1 | 4 | 5 |
| W4 | 2 | 4 | 5 |

$$\theta = 1, \qquad u_2, u_4 \text{ increase by } 1, \quad v_3 \text{ decreases by } 1.$$

> Uncovered reduced costs decrease by 1. Entries covered twice increase by 1. Entries covered once stay unchanged. All reduced costs remain nonnegative.

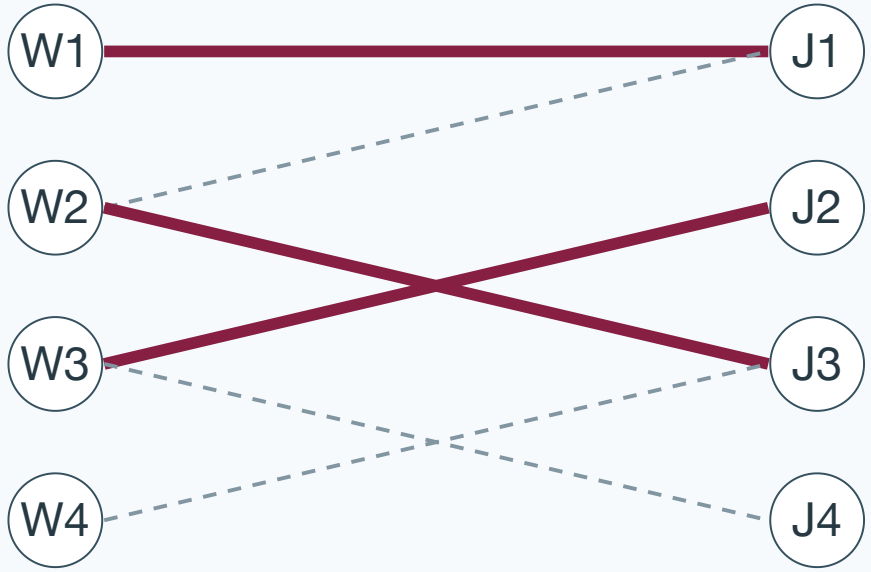
The original assignment costs do not change; only the dual variables and reduced costs change.

# After adjustment 1: still three pairs

**Reduced costs: original cost - worker dual variable - job dual variable**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | **0 ★** | 4 | 1 | 2 |
| **W2** | **0** | 3 | **0 ★** | 4 |
| **W3** | 2 | **0 ★** | 1 | **0** |
| **W4** | 1 | 3 | **0** | 4 |

$$u = (8, 8, 9, 9), \quad v = (2, 7, -1, 5),$$
$$\sum_i u_i + \sum_j v_j = 47.$$




Every line has reduced cost zero. Thick maroon lines are chosen pairs; dashed lines are other available pairs.

The new zero is W2–J1. Search again from W4: W4 → J3 → W2 → J1 → W1. Neither J2 nor J4 is reached.

# Your turn: find the second adjustment

## Current reduced costs

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | **0** | 4 | 1 | 2 |
| **W2** | **0** | 3 | **0** | 4 |
| **W3** | 2 | **0** | 1 | **0** |
| **W4** | 1 | 3 | **0** | 4 |

Current $u = (8, 8, 9, 9)$, $v = (2, 7, -1, 5)$.

Work for about 5 minutes.

1. The reached workers are S = {W1, W2, W4}; reached jobs are T = {J1, J3}. Which lines cover all zeros?
2. Find the smallest uncovered entry.
3. Update u and v. Which new zero appears?

# Adjustment 2 creates the missing route to job 4

Cover row W3 and columns J1, J3. Uncovered columns are J2, J4 in rows W1, W2, W4.

| Uncovered row | J2 | J4 |
|---|---|---|
| W1 | 4 | 2 |
| W2 | 3 | 4 |
| W4 | 3 | 4 |

$$\theta = 2, \qquad u = (10, 10, 9, 11), \qquad v = (0, 7, -3, 5).$$

W1–J4 becomes zero. The dual value increases from 47 to 49. Negative dual variables are allowed because the primal assignment constraints are equalities.

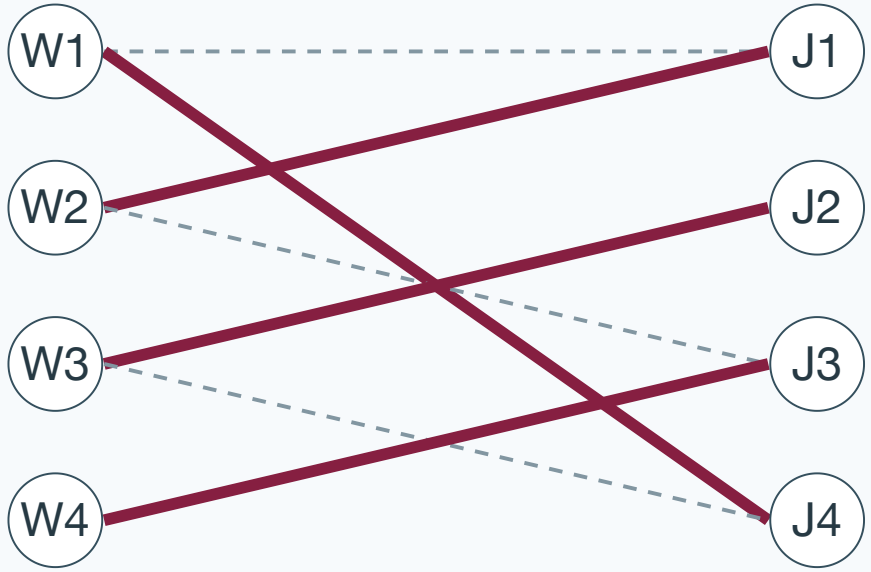
# Use the new alternating path to assign worker 4

## Final reduced costs

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | **0** | 2 | 1 | **0** |
| **W2** | **0** | 1 | **0** | 2 |
| **W3** | 4 | **0** | 3 | **0** |
| **W4** | 1 | 1 | **0** | 2 |

$$\mathrm{W4} \to \mathrm{J3} \to \mathrm{W2} \to \mathrm{J1} \to \mathrm{W1} \to \mathrm{J4}.$$

Remove W2–J3 and W1–J1. Add W4–J3, W2–J1, W1–J4. Keep W3–J2.




Every line has reduced cost zero. Thick maroon lines are chosen pairs; dashed lines are other available pairs.

# Original costs and the dual agree: 49

**Original costs; ★ marks the final assignment**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| W1 | 10 | 19 | 8 | 15 ★ |
| W2 | 10 ★ | 18 | 7 | 17 |
| W3 | 13 | 16 ★ | 9 | 14 |
| W4 | 12 | 19 | 8 ★ | 18 |

**Primal assignment cost**

$$15 + 10 + 16 + 8 = 49.$$

**Dual lower bound**

$$(10 + 10 + 9 + 11) + (0 + 7 - 3 + 5) = 49.$$

Every reduced cost is nonnegative, so the dual is feasible. Equal values prove optimality.

# The Hungarian adjustment in general

Maintain $R_{ij} = c_{ij} - u_i - v_j \geq 0$ and a matching on zero entries. Run alternating searches from unmatched workers.

> If an unmatched job is reached, augment the matching. If no such job is reached, let S and T be the reached workers and jobs.

$$\theta = \min_{i \in S,\ j \notin T} R_{ij} > 0.$$

Increase $u_i$ by $\theta$ for $i \in S$; decrease $v_j$ by $\theta$ for $j \in T$. Leave the other dual variables unchanged.

Repeat the search. Stop when the zero matching is perfect. A complete cost matrix is assumed here.

# Why the four kinds of entries change differently

| Entry location | Change in u + v | Change in reduced cost |
|---|---|---|
| **Reached worker; unreached job** | $+\theta$ | $-\theta$: creates a new zero |
| **Reached worker; reached job** | 0 | 0 |
| **Unreached worker; unreached job** | 0 | 0 |
| **Unreached worker; reached job** | $-\theta$ | $+\theta$ |

> A chosen pair has either both endpoints reached or both unreached, so its reduced cost stays zero. Some other old zeros can become positive; they are not all preserved.

Appendix: feasibility, matching preservation, progress, and optimality.

# Practice: one more complete assignment

## A new cost matrix

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 82 | 83 | 69 | 92 |
| **W2** | 77 | 37 | 49 | 92 |
| **W3** | 11 | 69 | 5 | 86 |
| **W4** | 8 | 9 | 98 | 23 |

Work for about 5 minutes.

Start from $u = (69, 37, 5, 8)$, $v = (0, 0, 0, 15)$.

1. Compute the reduced table.
2. Find a maximum zero matching and a cover.
3. Make one adjustment and give an optimal assignment.

# Practice answer: minimum cost 140

## Before the adjustment

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 13 | 14 | **0** | 8 |
| **W2** | 40 | **0** | 12 | 40 |
| **W3** | 6 | 64 | **0** | 66 |
| **W4** | **0** | 1 | 90 | **0** |

Cover rows W2, W4 and column J3. The smallest uncovered entry is 6.

$$u = (75, 37, 11, 8), \quad v = (0, 0, -6, 15).$$

Choose W1–J3, W2–J2, W3–J1, W4–J4.

$$69 + 37 + 11 + 23 = 140.$$

The dual value is also 140. All 16 reduced costs remain nonnegative.

Appendix: the complete updated table and alternating path.

# A different approach: resolve competition by bidding

The Hungarian method adjusts dual variables until a perfect matching on zero reduced-cost entries is possible. An **auction algorithm** instead lets unassigned workers compete for jobs.

> For this new example, maximize total benefit. Let $a_{ij}$ be worker i's benefit from job j, and $q_j$ a numerical job price used by the algorithm.

Start with every worker unassigned and every job price zero. A worker compares benefit minus job price. Increasing a contested job's price can make a different job more attractive.

These prices are algorithm variables, not additional payments in the original assignment objective.

# One bid: compare the best and second-best jobs

For an unassigned worker $i$, compare $a_{ij} - q_j$. Choose a best job $j^*$; let V be its net benefit and W the second-best net benefit.

$$q_{j^*} \leftarrow q_{j^*} + (V - W) + \varepsilon, \qquad \varepsilon > 0.$$

> Assign this worker to the chosen job. Its former owner, if any, becomes unassigned and bids later. Repeat until all workers are assigned.

The small positive $\varepsilon$ makes each bid strictly increase a price, including ties. This rule assumes at least two jobs; a one-job problem is immediate.

# Three bids settle a two-worker example

| Benefits | J1 | J2 |
|---|---|---|
| W1 | 8 | 6 |
| W2 | 7 | 4 |

Start at $q = (0, 0)$, choose $\varepsilon = 1/4$, and let W1 bid first.

| Bid | Net benefits before bid | Result |
|---|---|---|
| W1 | (8, 6) | J1 price becomes 2.25; W1 takes J1 |
| W2 | (4.75, 4) | J1 price becomes 3.25; W2 displaces W1 |
| W1 | (4.75, 6) | J2 price becomes 1.50; W1 takes J2 |

Final assignment: W1–J2 and W2–J1, benefit 6 + 7 = 13. The other assignment gives 8 + 4 = 12.

# How close is an auction assignment to optimal?

At termination, every assigned worker receives net benefit within $\varepsilon$ of its best available net benefit. This property is called $\varepsilon$-complementary slackness.

$$a_{i,\sigma(i)} - q_{\sigma(i)} \geq \max_j (a_{ij} - q_j) - \varepsilon.$$

Here $\sigma(i)$ is the job assigned to worker $i$. Summing over workers shows that the total benefit is within $n\varepsilon$ of the optimum.

> If benefits are integers and $n\varepsilon < 1$, the whole assignment is exactly optimal: two integer assignment values cannot differ by a positive amount below 1.

Appendix: the bound, why bids preserve it, and termination.

# What connects assignment algorithms to duality?

| Method | What it maintains | How it finishes |
|---|---|---|
| **Hungarian** | Feasible dual; zero reduced-cost matching | Perfect matching on zeros gives equal primal/dual values |
| **Auction** | Job prices; approximate complementary slackness | Complete assignment within $n\varepsilon$ of optimum |

> For hand calculations: write the original costs, current dual variables, reduced table, chosen pairs, and the reason for the next change. The final original objective value needs a certificate.

Next: use a spanning tree as a simplex basis for a general minimum-cost flow problem.

Appendix: proofs, extra practice, and model variations.

# Appendix: proofs and additional worked details

Why Hungarian adjustments are valid

Why the procedure reaches a perfect matching

Why fractions do not lower the optimum

Practice: the complete adjusted table

Auction bounds and termination

Unequal counts and maximization

# Dual feasibility and chosen pairs survive an adjustment

After a failed alternating search, there is no zero edge from S to a job outside T. Every entry used in the minimum θ is therefore strictly positive.

Those entries decrease by θ and remain nonnegative. Entries from workers outside S to jobs in T increase. The other two entry types are unchanged. Hence the updated dual is feasible.

If a matched job is reached, its matched worker is reached by the backward step. A reached matched worker was reached through its own matched job. Thus a chosen pair has both endpoints reached or neither.

> Every matched reduced cost stays zero. The smallest uncovered entry becomes zero, extending the possible alternating search.

# Why a perfect matching is eventually reached

For a fixed matching, start alternating searches from all unmatched workers. If no unmatched job is reached, each reached job is paired with one reached worker; S also contains the unmatched roots. Thus $|S| > |T|$.

An adjustment leaves every existing alternating-search edge zero and adds at least one edge from S to outside T. Continue the same search. It reaches a new job or an unmatched job.

With finitely many jobs, an augmenting path is eventually found. Exchanging its edges grows the matching by one while keeping every chosen reduced cost zero.

> After at most n augmentations, a perfect zero matching and a feasible dual have equal values. Efficient implementations maintain these searches and minima to achieve $O(n^3)$ time.

# Why the zero cover is minimum

Let a zero matching have maximum size k. Search from all unmatched workers. No unmatched job is reachable, or an alternating path would enlarge the matching.

The unreached worker rows together with reached job columns cover every zero. Otherwise a zero edge from a reached worker would lead to an unreached job.

$$|\text{cover}| = n - |S| + |T| = k.$$

Each line can cover at most one edge of any matching because matching edges have distinct endpoints. Therefore any zero cover has at least k lines, and this cover is minimum.

> This proves the matching–cover equality for this bipartite graph and explains how to construct the cover.

# Why an optimal whole assignment exists

Let X be any feasible fractional assignment. In its graph of positive entries, take a set S of workers and their neighboring jobs N(S).

$$|S| = \sum_{i \in S, j} x_{ij} \leq \sum_{j \in N(S), i} x_{ij} = |N(S)|.$$

A maximum matching missing a worker would have a failed alternating search with |S| > |N(S)|, contradicting this inequality. Hence the positive entries contain a perfect matching. Let P be its assignment matrix: 1 for each chosen pair and 0 otherwise.

Let α be the smallest selected entry. Subtract αP. If α < 1, divide the remainder by 1 - α to obtain another feasible fractional assignment, with fewer positive entries.

> Repeating gives X as a convex combination of whole assignments. At least one has cost no greater than X. Thus fractions cannot improve the minimum.

# Practice: all updated reduced costs

**After θ = 6**

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| **W1** | 7 | 8 | **0** | 2 |
| **W2** | 40 | **0** | 18 | 40 |
| **W3** | **0** | 58 | **0** | 60 |
| **W4** | **0** | 1 | 96 | **0** |

$$u = (75, 37, 11, 8),\ v = (0, 0, -6, 15).$$

Before adjustment, choose W1–J3, W2–J2, W4–J4; W3 is unmatched. The alternating search reaches W3, J3, W1.

The new W3–J1 zero directly reaches an unmatched job. Add it; the other three chosen pairs stay.

> Cost = 69 + 37 + 11 + 23 = 140. Dual sum = 131 + 9 = 140.

# Auction: derive the nε guarantee

Let OPT be the maximum total assignment benefit. Define $U_i = \max_j(a_{ij} - q_j)$. Then $U_i + q_j \geq a_{ij}$, so $\sum_i U_i + \sum_j q_j$ is an upper bound for the maximum-benefit assignment.

For the final assignment $\sigma$, approximate complementary slackness gives $a_{i,\sigma(i)} \geq U_i + q_{\sigma(i)} - \varepsilon$.

$$\sum_i a_{i,\sigma(i)} \geq \sum_i U_i + \sum_j q_j - n\varepsilon \geq \mathrm{OPT} - n\varepsilon.$$

After a bid, the chosen job's net benefit is W - ε; every other net benefit is at most W. A still-assigned worker's own job price does not change unless that worker is displaced. Other price increases cannot make alternative jobs more attractive.

> Thus approximate complementary slackness is preserved for every assigned worker.

# Auction: why bidding stops for a complete finite matrix

Each bid raises a price by at least ε. Once a job is assigned, it stays assigned, though its owner may change. A job that has never been assigned still has its initial price 0.

Suppose bidding continued forever. The number of assigned jobs can increase at most n times. Eventually it would remain fixed below n, with some unassigned job at price 0.

Let A be the largest absolute benefit. Every later bid is for an already assigned job. Its second-best net benefit W is at least -A, because an unassigned price-0 job remains available.

The newly assigned job price is $a_{ij}$ - W + ε ≤ 2A + ε. Finitely many prices cannot increase by at least ε infinitely often while staying bounded.

> This contradiction proves termination. Assumptions: finite benefits, all pairs allowed, and ε > 0.

# Unequal counts, prohibited pairs, and maximum benefit

| Variation | Modeling change |
| --- | --- |
| **More jobs than workers** | Add dummy workers with explicitly specified costs |
| **More workers than jobs** | Add dummy jobs representing non-assignment |
| **A prohibited worker–job pair** | Remove that variable and its graph edge |
| **Maximize total benefit $a_{ij}$** | Minimize $c_{ij}$ = M - $a_{ij}$, for a common M |

A complete assignment always uses n entries, so its transformed cost equals nM minus its original benefit.

> For prohibited pairs, a perfect matching may not exist. A failed search with no uncovered permitted entry proves infeasibility; do not take a minimum over an empty set.

# Additional practice: a zero matching after reductions

## New original costs

| Worker | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| W1 | 9 | 2 | 7 | 8 |
| W2 | 6 | 4 | 3 | 7 |
| W3 | 5 | 8 | 1 | 8 |
| W4 | 7 | 6 | 9 | 4 |

Use $u = (2, 3, 1, 4)$, $v = (3, 0, 0, 0)$.

The zero matching is W1–J2, W2–J1, W3–J3, W4–J4.

$$2 + 6 + 1 + 4 = 13 = \sum_i u_i + \sum_j v_j.$$

No adjustment is needed: a feasible dual and a complete zero matching already certify the answer.